

AI ALGORITHM PATTERNS · PATTERN 02

Convolution & CNNs: The Sliding Window That Sees

How a 3×3 grid of numbers, slid across an image, becomes face unlock, tumor detection, and a self-driving car's eyes.

Merit Point Academy · 30 min · press **S** notes · → next ·  to comment

AGENDA

Where we're going

- **0–5'** Warm-up + what a computer actually "sees"
- **5–14'** The kernel — play with a live convolution animation
- **14–20'** Stack it into a CNN, then run real Python
- **20–26'** Where CNNs hide: your phone → hospitals → self-driving · and the RNN story
- **26–30'** CNN or RNN? Spot the pattern & wrap-up

By the end: explain what a convolution does, why CNNs see edges before objects, and when you'd reach for a CNN vs. an RNN.

WARM-UP · ACTIVATE

Quick check · tap an answer

A computer 'looks' at a photo of a cat. What does it actually receive?

A tiny description in English, like 'a cat on a sofa'

A grid of numbers — one brightness value per pixel

The shapes and objects, already separated

Nothing — computers can't process photos without the internet

An image is a grid of numbers

- A grayscale photo is a table of **brightness values**: 0 = black, 255 = white. A 1000×1000 photo = **a million numbers**.
- Color = three grids stacked (red, green, blue channels).
- Nowhere in those numbers does it say "edge", "whisker", or "cat" — **meaning has to be computed**.
- Question of the day: **what computation turns pixels into meaning?**

a tiny "image"

0	0	255	255
0	0	255	255
0	0	255	255
0	0	255	255

↑ a vertical edge, hiding in numbers

Spot it yourself: in the tiny image above, where does dark meet bright? That boundary — the **edge** — is the first thing worth detecting. Everything else is built on it.

Convolution = a sliding window with weights

Take a tiny grid of weights — a **kernel** (say 3×3). Slide it across the image. At every stop: **multiply each pixel by its weight, add them up, write one number down.**

Place

kernel over a 3×3 patch

Multiply

pixel $\times$ weight, all 9 pairs

Sum

→ one output number

Slide

one pixel over, repeat

Pattern series connection: Pattern 01 was *"keep only the best K."* Pattern 02 is *"slide a small window over big data"* — the same sliding-window idea behind moving averages and (spoiler) how attention scans text. A high output number means **"the thing this kernel describes is HERE."**

HANDS-ON · SLIDE THE WINDOW

Try it: convolve an image, live

Press ► **Play** and watch the 3×3 kernel scan the image — each stop multiplies, sums, and writes one output cell. Then **paint your own pixels**, switch kernels, and see what each one detects.

INPUT IMAGE (8×8) · CLICK PIXELS TO PAINT

1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0

Vertical edge

Horizontal edge

Diagonal

Smiley

X

Clear

KERNEL (3×3 FILTER)

1	0	-1
1	0	-1
1	0	-1

output = sum of the 9 products

1×1 + 1×0 + 1×(-1)

1×1 + 1×0 + 1×(-1)

1×1 + 1×0 + 1×(-1)

= 0

Vertical edge

Horizontal edge

Blur (average)

Sharpen

OUTPUT FEATURE MAP (6×6)

0					

Window: 1 / 36 · row 1, col 1

⏮ First

⏪ Prev

▶ Play

Next ▶

⏭ Last

Speed

CHECK · AFTER THE ANIMATION

Quick check · tap an answer

The vertical-edge kernel slides over a region where **ALL** pixels have the same brightness. What does it output there?

A big positive number

A big negative number

Exactly 0 — the +1 column and the -1 column cancel out

It crashes — kernels need edges to work

A CNN = layers of learned kernels

A **Convolutional Neural Network** is that animation, repeated: convolve → keep what fired → convolve the result again, dozens of layers deep.

Early layers

Kernels that fire on **edges & lines** — like the ones you just played with.

Middle layers

Combine edges into **textures & parts**: fur, an eye, a wheel, a corner.

Late layers

Combine parts into **whole objects**: "cat 0.96 · dog 0.03".

The twist that makes it AI: nobody writes the kernel numbers. They start random, and **training tunes them** — the network invents its own edge detectors because they're genuinely useful for the task. That's the "learning" in deep learning.

Write convolution from scratch — 12 lines

The whole operation from the animation, in pure Python — no libraries, runs instantly. Run it, find the edge in the output, then try the challenges.

```
# Convolution from scratch - no libraries needed.
# A 6x6 image: 1 = bright, 0 = dark. Left half bright -> one vertical edge.
img = [
    [1, 1, 1, 0, 0, 0],
    [1, 1, 1, 0, 0, 0],
    [1, 1, 1, 0, 0, 0],
    [1, 1, 1, 0, 0, 0],
    [1, 1, 1, 0, 0, 0],
    [1, 1, 1, 0, 0, 0],
]

# The vertical-edge kernel from the animation
kernel = [
    [1, 0, -1]
```



Run



Explain



Debug



Improve

Python loads on first Run

► Run the code to see the output here.

AI Assistant



Haiku · fastest



Hi! Edit the code and hit **Run**. Stuck? I can **Explain**, **Debug**, or **Improve** it — or just ask me below.

Ask

You used a CNN before breakfast



Face unlock

A CNN turns your face into a signature of numbers and matches it — edges → features
→ "it's you", in milliseconds, on the phone itself.



Portrait mode

A CNN labels every pixel **person** / **background** so the camera knows what to blur.
Night mode & sharpening: convolutions too.



Photo search

Type "dog" into your photos app — a CNN already tagged every image. No human ever labeled your library.

All three run the pattern you just wrote in 12 lines of Python — just with **millions of learned kernel weights** instead of one hand-made kernel.

Higher stakes: hospitals & self-driving cars

- **Medical imaging:** CNNs scan X-rays, CT slices, and skin photos for tumors — in several studies they match or beat radiologists at narrow detection tasks. Same pattern: suspicious texture = a kernel firing.
- **Self-driving perception:** every camera frame goes through CNNs that find lanes, cars, pedestrians, signs — **this is the "300 detected objects" that Pattern 01's Top-K then ranks.**

Your instructor's own research world: LiDAR–camera fusion and bridge-inspection automation both lean on CNN perception.



Camera frame → CNN → objects → Top-K → plan
Two patterns from this series, chained in one real system.

Patterns compose: **convolution finds the objects, Top-K decides which ones matter.** Real AI systems are pipelines of patterns like these.

And what's an RNN?

CNNs are built for **grids** (images). A **Recurrent Neural Network** is built for **sequences** — text, audio, anything that arrives in order.

Read one step

a word / audio slice

Update memory

the "hidden state"

Carry it forward

repeat for the next step

Like reading a sentence: you don't re-read from the start at every word — you carry a running summary in your head. That summary is the RNN's hidden state.

Two patterns, two data shapes: CNN = a window sliding over **space**. RNN = a memory running over **time**. Before 2017, RNNs powered translation, speech recognition, and autocomplete.

PLOT TWIST · WHY YOU HEAR LESS ABOUT RNNs NOW

Why Transformers replaced RNNs

RNN's two weaknesses

- 1 · **One step at a time.** Word 500 can't be processed before words 1–499 — painfully slow to train on GPUs built for parallel work.
- 2 · **Long-distance forgetting.** By the end of a long paragraph, the running memory has squeezed out the beginning.

Attention's fix (2017)

Let **every word look at every other word directly** — all at once, no running memory, fully parallel. The paper: *"Attention Is All You Need."*

The result is the **Transformer** — the **T in ChatGPT**. RNNs still matter (and their ideas echo in newer models), but attention won the language war. **How attention scans a sequence is exactly Pattern 03** — next in this series.

CNN or RNN? Look at the shape of your data

Grid-shaped?

Photos, medical scans, game boards, spectrograms → **CNN** (nearby pixels relate).

Sequence-shaped?

Text, audio, sensor streams, DNA → **RNN family, or today a Transformer.**

Both / unusual?

Video = grid + time. Molecules = graphs. Architectures mix & match — the data's shape decides.

This instinct — "what's the shape of my data?" — is a genuine research skill. You'll use it when you build your first neural network in AI Toolbox Class 13, and in every project after.

Quick check · [tap an answer](#)

Which pairing correctly matches the architecture to what it's built for?

CNN for sequences like text · RNN for grids like images

CNN for grids like images · RNN for sequences like text and audio

Both only work on black-and-white images

CNN and RNN are two names for the same network

WRAP-UP

Three things to keep

- An image is a **grid of numbers** — a convolution slides a small weighted window over it, and fires where the image matches the kernel's pattern
- A **CNN stacks learned kernels**: edges → textures → parts → objects. Nobody writes the kernels — training does
- **CNN for grids, RNN for sequences** — with attention/Transformers as the modern heir to RNNs

Exit ticket: name one moment today when a CNN almost certainly looked at pixels for you. Post it in .

Next pattern · Two Pointers & Sliding Window — how attention scans a sequence.